

Daws86s-Interview Preparation Series(DevSecOps Shell-Day4)

PHASE 1: FOUNDATION

What is Shell? What is Bash?

- Shell = interface between user and OS
- Bash = most popular shell (Bourne Again Shell)
- Shell scripting = automation language for Linux

Show practically:

```
echo $SHELL  
echo $0
```

Explain:

- \$SHELL → default shell
- \$0 → current shell/session

"Shell scripting is used to automate repetitive administrative tasks."

PHASE 1: FOUNDATION

Environment Setup

Topics:

01

Ubuntu installation / WSL

02

Terminal basics

03

Linux directory structure

Commands:

```
pwd
ls
cd /
ls /etc
ls /var/log
```

Explain:

| **/etc** → configs

| **/var/log** → logs

| **/home** → users

PHASE 2: BASIC COMMANDS

General Purpose Commands

Commands with examples:

```
ls -l  
ls -a  
pwd  
mkdir test  
touch file.txt  
cp file.txt file2.txt  
mv file2.txt backup.txt  
rm backup.txt
```

Explain:

-l → long listing

-a → hidden files

Help system:

```
man ls  
ls --help
```

PHASE 3: REDIRECTION

STDIN, STDOUT, STDERR

Explain clearly with table:

Stream	Meaning
0	STDIN
1	STDOUT
2	STDERR

Examples:

```
ls > out.txt
ls wrongfile 2> error.txt
ls &> all.txt
```

Real-world applications:

Logs

Error tracking

CI/CD output capture

PHASE 4: FILE READING COMMANDS

Reading Files

Commands:

```
cat file.txt  
less file.txt  
head -n 5 file.txt  
tail -n 10 file.txt  
tail -f /var/log/syslog
```

Range of lines:

```
sed -n '5,10p' file.txt
```

PHASE 5: GREP

grep – Search Engine of Linux

Basic grep:

```
grep error logfile.log  
grep -i error logfile.log  
grep -n error logfile.log  
grep -v error logfile.log
```

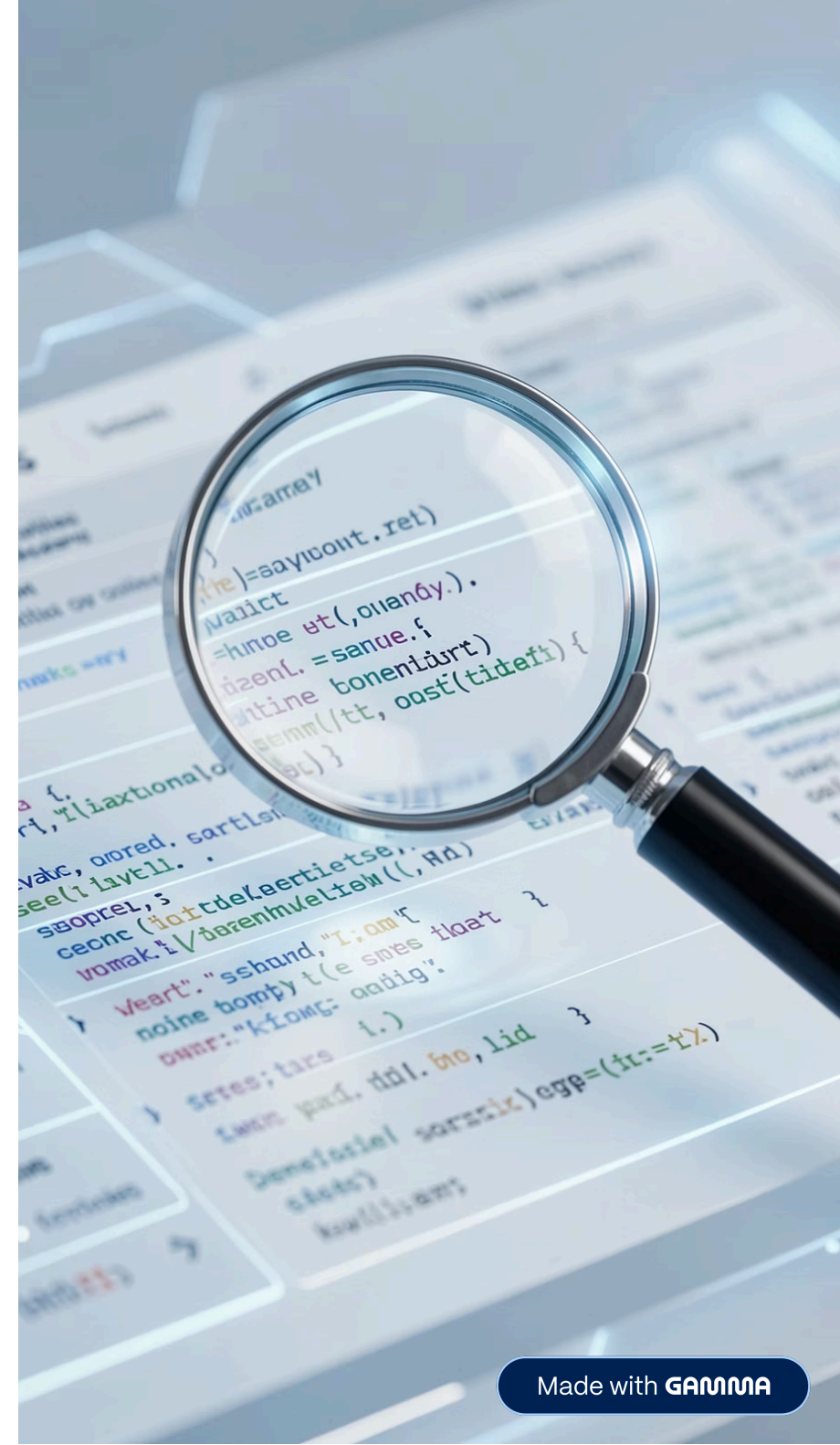
Recursive grep:

```
grep -r "DB_HOST" /etc
```

Production use:

Searching errors in logs

Finding config values



PHASE 6: CUT COMMAND

cut – Column Extraction

Examples:

```
cut -d ":" -f1 /etc/passwd  
cut -d "," -f2 data.csv
```

Explain:

-d → delimiter

-f → field

PHASE 7: AWK

awk – Power Tool

Basics:

```
awk '{print $1}' file.txt  
awk '{print $1,$3}' file.txt
```

With condition:

```
awk '$3 > 50 {print $1,$3}' marks.txt
```

With variables:

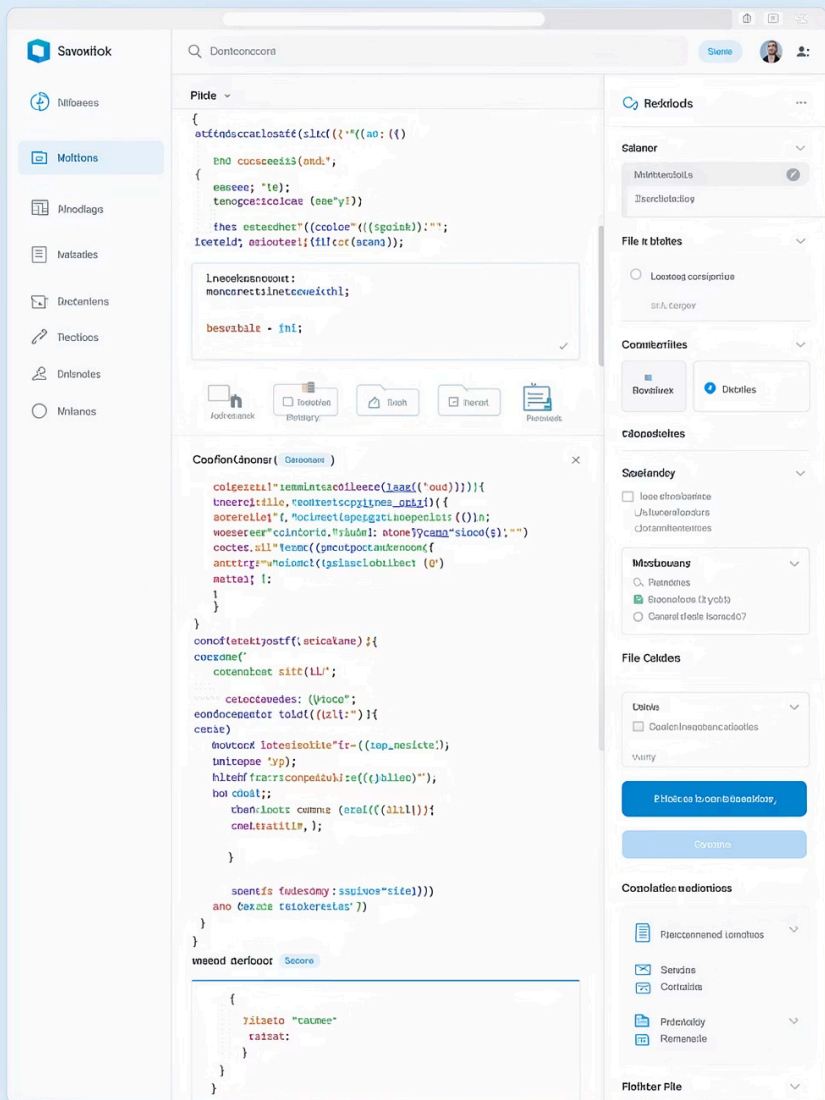
```
awk -v limit=80 '$2 > limit {print $1}' marks.txt
```

Arithmetic:

```
awk '{sum+=$2} END {print sum}' file.txt
```

Key insight: awk is like Excel inside Linux





PHASE 8: SED

sed – Editing Engine

Replace:

```
sed 's/old/new/' file.txt  
sed 's/old/new/g' file.txt
```

Delete lines:

```
sed '5d' file.txt  
sed '2,4d' file.txt
```

Insert:

```
sed '2i Hello' file.txt
```

PHASE 8: SED + REGEX

sed – Advanced Patterns (VERY IMPORTANT)

Advanced sed with regex:

```
sed '/^error/d' logfile  
sed '/^$/d' file.txt  
sed 's/[0-9]//g' file.txt
```

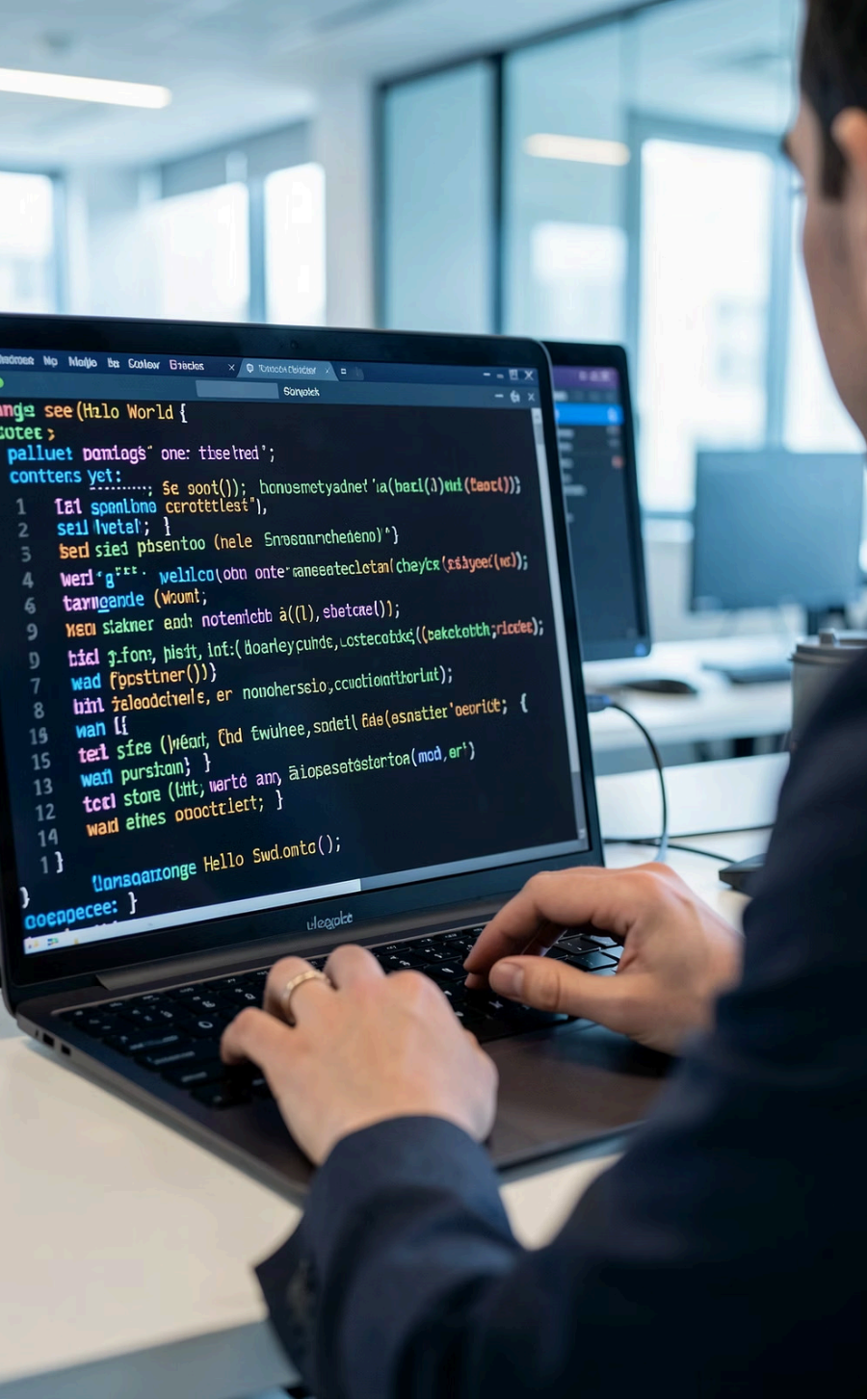
Explain regex patterns:

| ^ → start of line

| \$ → end of line

| [] → character range





PHASE 9: BASH SCRIPTING BASICS

First Script

Your first script:

```
#!/bin/bash  
echo "Hello World"
```

Explain:

shebang (**#!/bin/bash**) tells the system to use bash interpreter

execution permission needed

Commands:

```
chmod +x script.sh  
./script.sh
```



PHASE 9: BASH SCRIPTING BASICS

Variables & User Input

Variables:

```
name="Keerti"  
echo $name
```

Read input from user:

```
read user  
echo "Hello $user"
```

PHASE 9: BASH SCRIPTING BASICS

Command Line Arguments

Command line arguments:

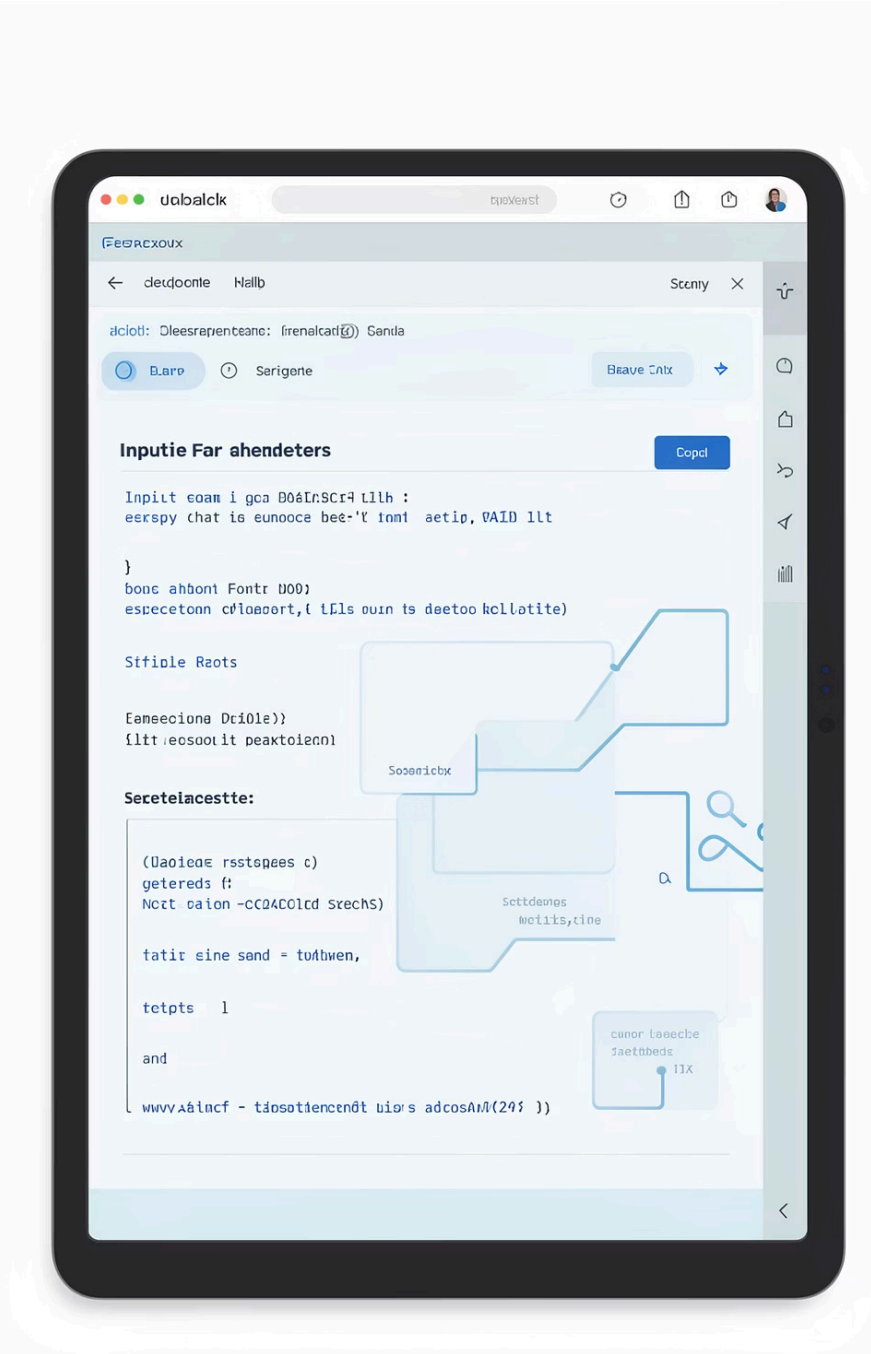
```
echo $1
echo $2
echo $#
echo $@
```

Interview confusion point:

```
@ vs *
```

Run example:

```
./script.sh DevOps Linux
```



PHASE 10: STRING OPERATIONS

String Manipulation

String operations:

```
str="DevOpsEngineer"
```

```
echo ${#str}
```

```
echo ${str:0:6}
```

```
echo ${str/Engineer/Admin}
```

Explain:

`${#str}` → length of string

`${str:0:6}` → substring from position 0, length 6

`${str/Engineer/Admin}` → replace Engineer with Admin



PHASE 11: ARITHMETIC & CASE

Arithmetic Operations

Arithmetic operations:

```
a=10
```

```
b=5
```

```
echo "Addition: $((a+b))"
```

```
echo "Subtraction: $((a-b))"
```

```
echo "Multiplication: $((a*b))"
```

```
echo "Division: $((a/b))"
```

PHASE 11: ARITHMETIC & CASE

Case Statement

Case statement:

```
case $1 in
start)
  echo "Service starting"
  ;;
stop)
  echo "Service stopping"
  ;;
restart)
  echo "Service restarting"
  ;;
*)
  echo "Usage: $0 {start | stop | restart}"
  ;;
esac
```



PHASE 12: CONDITIONS & TEST

If-Else Conditions

Basic if-else:

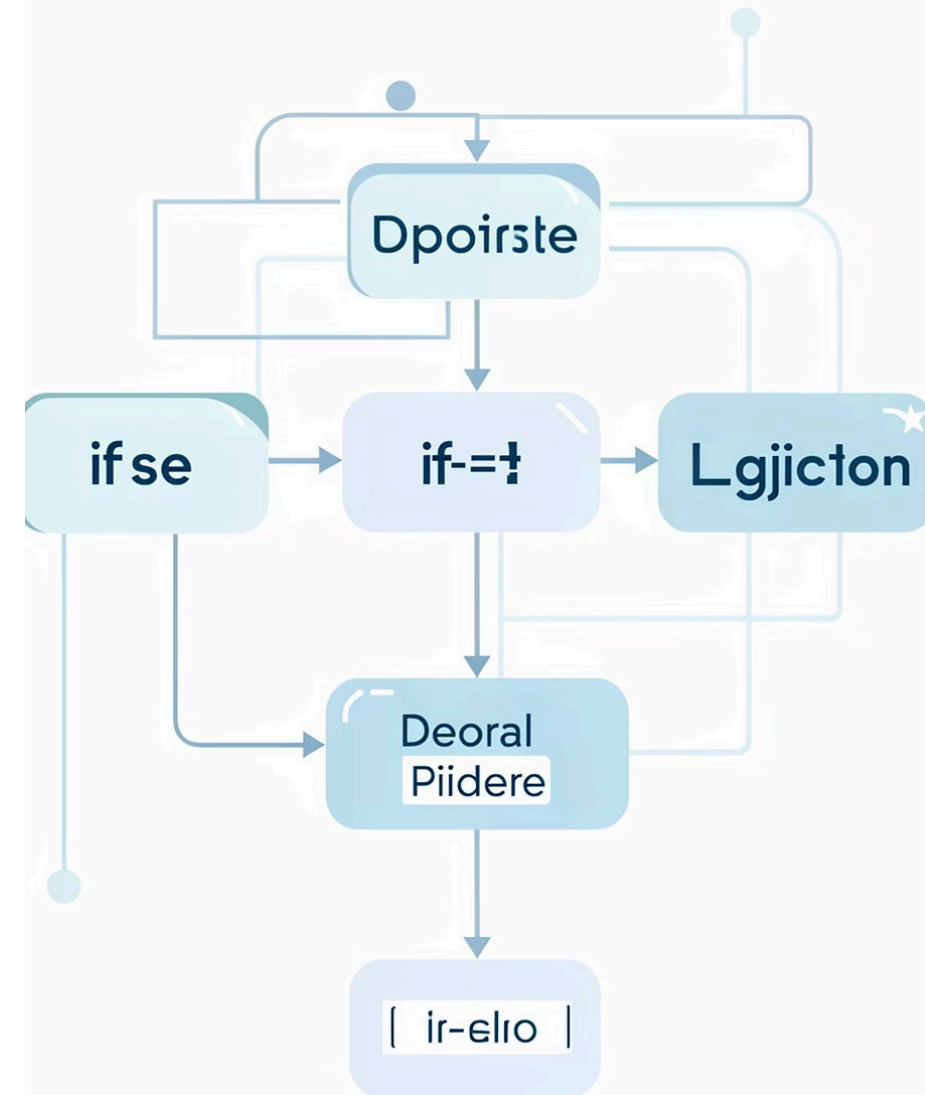
```
if [ $a -gt $b ]; then  
    echo "A is greater"  
fi
```

File checks:

```
[ -f file.txt ] # file exists  
[ -d /etc ]     # directory exists
```

Logical operators:

```
if [ -f file.txt ] && [ -r file.txt ]; then  
    echo "File exists and readable"  
fi
```



```
for loop;  
until her ten p)  
astitars,  
no 10e ({ifb, }  
Lwoer ()') {  
tod 160, "_f(, );  
falr "{;  
}
```

PHASE 13: LOOPS

For Loop

For loop:

```
for i in 1 2 3 4 5  
do  
    echo "Number: $i"  
done
```

For loop with arguments:

```
for pkg in "$@"  
do  
    echo "Installing $pkg"  
done
```

PHASE 13: LOOPS

While Loop

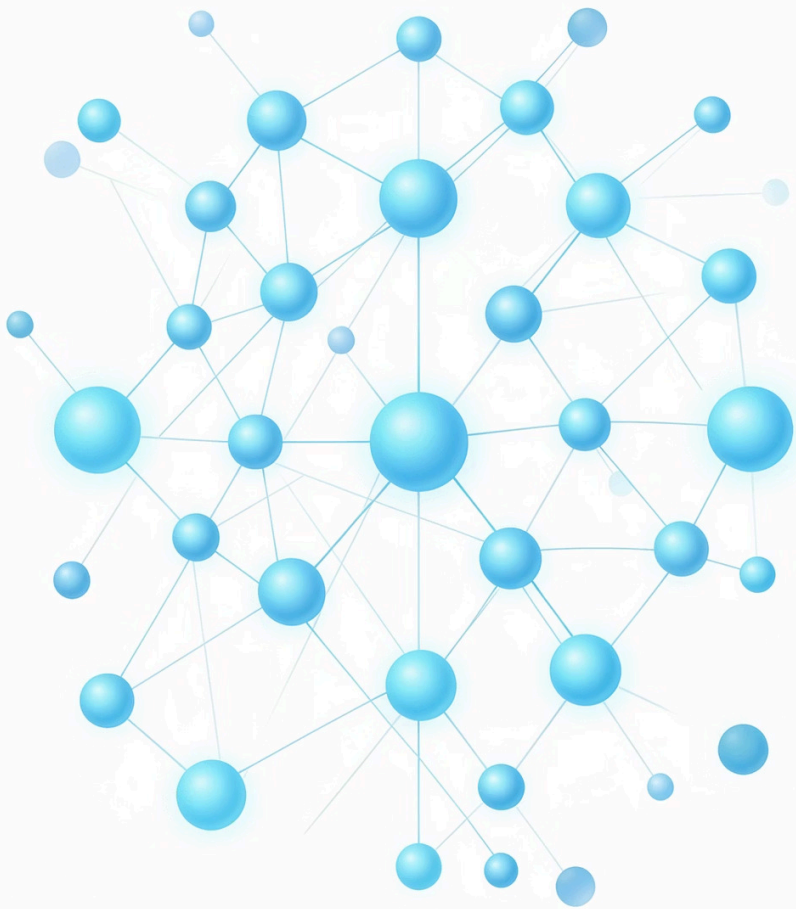
While loop:

```
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    ((count++))
done
```

Read file line by line:

```
while read line
do
    echo $line
done < file.txt
```





PHASE 14: ARRAYS

Array Basics

Array:

```
servers=("server1" "server2" "server3")
```

```
for s in "${servers[@]}"  
do  
    echo $s  
done
```

Access array elements:

```
echo ${arr[0]}    # first element  
echo ${arr[@]}    # all elements
```

PHASE 14: ARRAYS

Associative Arrays

Associative array:

```
declare -A user  
user[name]="Keerti"  
user[role]="DevOps"  
  
echo ${user[name]}  
echo ${user[role]}
```

Use case: Store key-value pairs like configuration data

PHASE 15: FUNCTIONS

Function Basics

Basic function:

```
hello() {  
  echo "Hello $1"  
}  
  
hello DevOps
```

Function with return value:

```
add() {  
  echo $(( $1 + $2 ))  
}  
  
result=$(add 10 20)  
echo "Sum: $result"
```

PHASE 15: FUNCTIONS

Exit Status

Exit status:

```
ls /wrongpath  
echo "Exit status: $?"
```

Explain:

\$? → exit status of last command

0 = success

non-zero = failure

Use in conditions:

```
if command; then  
  echo "Success"  
else  
  echo "Failed"  
fi
```

PHASE 16: SCHEDULING

Cron Jobs

Cron basics:

```
crontab -e
```

Cron syntax:

```
*/5 * * * * /path/script.sh
```

Explain:

***/5** → every 5 minutes

*** * * * *** → minute hour day month weekday

Real-world example - Disk usage alert:

```
#!/bin/bash
usage=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
if [ $usage -gt 80 ]; then
    echo "Disk usage is high: $usage%"
fi
```

Cron entry:

```
*/5 * * * * /path/disk.sh
```

PHASE 16: SCHEDULING

Memory Monitoring Script

Memory monitoring:

```
#!/bin/bash

free_mem=$(free -m | awk 'NR==2 {print $4}')

if [ $free_mem -lt 500 ]; then
    echo "Low memory: $free_mem MB"
fi
```

Use case: Alert when free memory drops below threshold